

ĐIỀU KHIỂN GPIO TRÊN STM32 QUA THANH GHI

1. Bản chất của Lập trình nhúng

Trong lập trình phần mềm thông thường, dữ liệu được xử lý trên RAM. Tuy nhiên, trong **Lập trình nhúng**, mọi hành động điều khiển phần cứng thực chất đều quy về một nguyên lý cốt lõi: **Đọc và Ghi dữ liệu vào các Thanh ghi (Registers)**.

- **Thanh ghi là gì?** Là các ngăn nhớ có địa chỉ vật lý xác định, nằm bên trong các vi mạch ngoại vi (như GPIO, UART, Timer...). Mỗi bit trong thanh ghi hoạt động như một "công tắc" phần cứng để bật/tắt hoặc cấu hình một tính năng cụ thể.
 - **Thao tác ĐỌC/GHI:**
 - **GHI (Write):** Đưa dữ liệu (0 hoặc 1) vào thanh ghi để ra lệnh cho phần cứng. Ví dụ: Cấp điện áp 3.3V ra một chân vi điều khiển để làm sáng LED.
 - **ĐỌC (Read):** Lấy dữ liệu từ thanh ghi để CPU biết trạng thái thực tế của phần cứng. Ví dụ: Kiểm tra xem có dòng điện truyền vào chân vi điều khiển khi người dùng nhấn nút hay không.
-

2. Tổng quan về bộ GPIO trên STM32

GPIO (General Purpose Input/Output) là bộ ngoại vi cơ bản và quan trọng nhất, cho phép vi điều khiển tương tác với các linh kiện điện tử bên ngoài.

A. Chức năng của GPIO

- **Output (Đầu ra):** Xuất mức điện áp (thường là 3.3V hoặc 0V) để điều khiển các cơ cấu chấp hành như LED, Relay, còi Buzzer.
- **Input (Đầu vào):** Nhận biết mức điện áp từ bên ngoài truyền vào để đọc trạng thái của nút nhấn, công tắc hành trình, hoặc cảm biến số.

B. Phân loại và Cách đặt tên

Trong vi điều khiển STM32, các chân GPIO không đứng rời rạc mà được nhóm thành các **Port** (Cổng).

- Các Port được đặt tên theo bảng chữ cái: **GPIOA, GPIOB, GPIOC, GPIOD, GPIOE...**
- Mỗi Port quản lý tối đa **16 chân vật lý**, được đánh số thứ tự từ 0 đến 15.
- **Quy tắc gọi tên:** **P** + **Tên Port** + **Số thứ tự**.
 - Ví dụ 1: **GPIOA** quản lý 16 chân từ **PA0** đến **PA15**.
 - Ví dụ 2: **GPIOD** quản lý 16 chân từ **PD0** đến **PD15**.

Khi lập trình, mỗi bit trong thanh ghi cấu hình của một Port sẽ tương ứng trực tiếp với số thứ tự của chân đó trên mạch thực tế.

3. Bộ 3 thanh ghi cốt lõi điều khiển GPIO

Để điều khiển một chân GPIO, bắt buộc phải tuân thủ quy trình: **Cấu hình trước, Thao tác sau**. Dưới đây là 3 thanh ghi quan trọng nhất:

3.1. Thanh ghi MODER (Mode Register) - Cấu hình chức năng (Có địa chỉ offset = 0x00)

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Trước khi thực hiện lệnh đọc hay ghi, vi điều khiển cần biết chân đó đang làm nhiệm vụ gì.

- **Chức năng:** Cấu hình hướng dữ liệu (Input hay Output) cho chân GPIO.
- **Quy tắc:** Mỗi chân sử dụng **2 bit** trong thanh ghi **MODER**.
 - **00**: Cấu hình chân làm **INPUT** (Đầu vào).
 - **01**: Cấu hình chân làm **OUTPUT** (Đầu ra).
- *Ví dụ:* Để cấu hình chân PD12 là Output, cần set bit số 24 và 25 của thanh ghi **GPIOA_MODER** thành giá trị **01**.

3.2. Thanh ghi ODR (Output Data Register) - Điều khiển đầu ra (Có địa chỉ Offset = 0x14)

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..E and H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).

Thanh ghi này chỉ có tác dụng khi chân đã được cấu hình là OUTPUT trong thanh ghi **MODER**.

- **Chức năng:** Xuất mức điện áp mong muốn ra chân vi điều khiển.
- **Quy tắc:** Mỗi chân sử dụng **1 bit** trong thanh ghi **ODR**.
 - Ghi **1**: Chân vi điều khiển xuất mức điện áp cao (3.3V).
 - Ghi **0**: Chân vi điều khiển xuất mức điện áp thấp (0V).
- *Ví dụ:* Ghi giá trị 1 vào bit số 12 của thanh ghi **GPIOA_ODR** sẽ làm chân PD12 có điện 3.3V.

3.3. Thanh ghi IDR (Input Data Register) - Đọc trạng thái đầu vào (Có địa chỉ Offset = 0x10)

8.4.5 GPIO port input data register (GPIOx_IDR) (x = A..E and H)

Address offset: 0x10

Reset value: 0x0000 XXXX (where X means undefined)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y = 0..15)

These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

Thanh ghi này dùng để giám sát khi chân đã được cấu hình là INPUT trong thanh ghi **MODER**.

- **Chức năng:** Đọc trạng thái điện áp thực tế đang cấp vào chân vi điều khiển.
- **Quy tắc:** Mỗi chân sử dụng **1 bit** trong thanh ghi **IDR**.
 - Đọc được **1**: Đang có mức điện áp cao (3.3V) cấp vào chân.
 - Đọc được **0**: Đang có mức điện áp thấp (0V) cấp vào chân.

Thanh ghi	Chức năng	Phân bổ bit	Mục đích chính
MODER	Cấu hình	2 bit / 1 chân	Xác định chân là Input hay Output
ODR	Ghi (Write)	1 bit / 1 chân	Xuất tín hiệu 0 hoặc 1 ra ngoài
IDR	Đọc (Read)	1 bit / 1 chân	Lấy giá trị 0 hoặc 1 từ ngoài vào

4. Quy trình tra cứu và tính toán địa chỉ thanh ghi

Để thao tác với thanh ghi, ta cần xác định địa chỉ tuyệt đối của nó thông qua 3 bước:

Bước 1: Tìm Base Address trong Datasheet

- Mở tài liệu Datasheet của dòng vi điều khiển đang sử dụng.
- Tìm đến phần *Memory Mapping*.
- Xác định dải địa chỉ được cấp phát cho ngoại vi cần sử dụng và lấy địa chỉ bắt đầu.

**Table 10. STM32F411xC/xE
register boundary addresses (continued)**

Bus	Boundary address	Peripheral
AHB2	0x5000 0000 - 0x5003 FFFF	USB OTG FS
AHB1	0x4002 6800 - 0x4FFF FFFF	Reserved
	0x4002 6400 - 0x4002 67FF	DMA2
	0x4002 6000 - 0x4002 63FF	DMA1
	0x4002 5000 - 0x4002 4FFF	Reserved
	0x4002 3C00 - 0x4002 3FFF	Flash interface register
	0x4002 3800 - 0x4002 3BFF	RCC
	0x4002 3400 - 0x4002 37FF	Reserved
	0x4002 3000 - 0x4002 33FF	CRC
	0x4002 2000 - 0x4002 2FFF	Reserved
	0x4002 1C00 - 0x4002 1FFF	GPIOH
	0x4002 1400 - 0x4002 1BFF	Reserved
	0x4002 1000 - 0x4002 13FF	GPIOE
	0x4002 0C00 - 0x4002 0FFF	GIOD
	0x4002 0800 - 0x4002 0BFF	GPIOC
	0x4002 0400 - 0x4002 07FF	GPIOB
	0x4002 0000 - 0x4002 03FF	GPIOA

Bước 2: Tìm Offset Address trong Reference Manual

- Mở tài liệu Reference Manual.
- Tìm đến chương viết về ngoại vi đó (Ví dụ: General-purpose I/Os).
- Xem phần mô tả các thanh ghi (*Register description*) để lấy giá trị Address offset.

Bước 3: Cộng Hexadecimal (Hệ cơ số 16)

- Thực hiện phép cộng Base Address và Offset Address để ra được địa chỉ tuyệt đối cuối cùng theo công thức:

$$\text{Địa chỉ thanh ghi} = \text{Địa chỉ Cơ sở (Base Address)} + \text{Offset Address}$$

5. Quy trình Lập trình và Code mẫu C

Yêu cầu bài toán:

1. Cấu hình chân **PD12** làm Output để điều khiển LED.
2. Cấu hình chân **PA0** làm Input để đọc trạng thái nút nhấn.

CHÚ Ý QUAN TRỌNG: Trước khi sử dụng bất kỳ bộ ngoại vi nào, việc đầu tiên bắt buộc phải làm là sử dụng RCC để cấp Clock (xung nhịp) cho bộ ngoại vi đó hoạt động.

```
#define GPIOA_BASE_ADDR 0x40020000
#define GPIOD_BASE_ADDR 0x40020C00

// =====
// 1. KHỞI TẠO NÚT NHẤN (CHÂN PA0 - INPUT)
// =====
void button_init() {
    // Sử dụng RCC để cấp clock cho GPIOA
    __HAL_RCC_GPIOA_CLK_ENABLE();

    // Cấu hình PA0 là INPUT (Reset bit 0, 1 về 00)
    uint32_t* GPIOA_MODER = (uint32_t*)(GPIOA_BASE_ADDR + 0x00);
    *GPIOA_MODER &= ~(0b11 << 0);
}

// Hàm đọc trạng thái nút nhấn
int button_get_state() {
    uint32_t* GPIOA_IDR = (uint32_t*)(GPIOA_BASE_ADDR + 0x10);
    int button_state;

    // Trích xuất và kiểm tra bit số 0 của thanh ghi IDR
    if ((*GPIOA_IDR >> 0) & 0b1) == 1) {
        button_state = 1; // Nút đang được nhấn (mức 1)
    } else {
        button_state = 0; // Nút không nhấn (mức 0)
    }

    return button_state;
}

// =====
// 2. KHỞI TẠO LED (CHÂN PD12 - OUTPUT)
```

```
// =====
void led_init() {
    // Sử dụng RCC để cấp clock cho GPIO
    __HAL_RCC_GPIOA_CLK_ENABLE();

    // Cấu hình PD12 là OUTPUT (Set bit 24, 25 thành 01)
    uint32_t* GPIOA_MODER = (uint32_t*)(GPIOA_BASE_ADDR + 0
x00);
    *GPIOA_MODER &= ~(0b11 << 24); // Xóa dữ liệu cũ
    *GPIOA_MODER |= (0b01 << 24); // Ghi giá trị 01
}

// Hàm điều khiển bật/tắt LED
void led_control(int on_off) {
    uint32_t* GPIOA_ODR = (uint32_t*)(GPIOA_BASE_ADDR + 0x1
4);

    if (on_off == 1) {
        *GPIOA_ODR |= (1 << 12); // Ghi 1 vào bit 12 -> LE
D ON
    } else {
        *GPIOA_ODR &= ~(1 << 12); // Ghi 0 vào bit 12 -> LE
D OFF
    }
}
}
```

6. Bài tập thực hành

Sử dụng hai hàm `led_control(int on_off)` và `button_get_state()` đã xây dựng ở các phần trước để giải quyết các bài toán điều khiển dưới đây. (sử dụng `HAL_Init()`, và `HAL_Delay(ms)` để tạo khoảng thời gian chờ)

Bài tập 1: Điều khiển LED trực tiếp (Mức cơ bản)

- **Yêu cầu:** Viết chương trình thực thi trong vòng lặp `while(1)`. Khi người dùng nhấn giữ nút ở chân PA0, đèn LED ở chân PD12 bật sáng. Khi người dùng nhả nút, đèn LED ngay lập tức tắt.

Bài tập 2: Nhấn nhả bật/tắt (Toggle State)

- **Yêu cầu:** Viết chương trình để nút nhấn hoạt động như một công tắc luân phiên. Nhấn nút một lần (rồi nhả ra), LED sáng. Nhấn nút lần thứ hai (rồi nhả ra), LED tắt.
- **Gợi ý:** Sinh viên cần sử dụng biến trạng thái để lưu lại giá trị của LED và viết thêm thuật toán chống dội phím (debounce) bằng phần mềm.

Bài tập 3: Chớp nháy theo điều kiện (Conditional Blinking)

- **Yêu cầu:** Nếu nút không được nhấn, LED ở trạng thái tắt.
 - Nếu nút đang được nhấn và giữ nguyên, LED chớp nháy liên tục với chu kỳ 500ms (Sáng 500ms - Tắt 500ms).
 - Ngay khi nhả nút, quá trình chớp nháy dừng lại và LED quay về trạng thái tắt.

Bài tập 4: Đếm số lần tác động (Counter)

- **Yêu cầu:** Thiết kế hệ thống đếm số lần thao tác của người dùng:
 - Mỗi lần người dùng bấm và nhả nút nhấn sẽ được hệ thống tính là 1 lần hợp lệ.
 - Khi bộ đếm đạt đủ 5 lần nhấn, bật sáng LED.
 - Tiếp tục nhấn thêm 5 lần nữa (tổng cộng 10 lần), tắt LED và reset giá trị bộ đếm về 0 để bắt đầu chu trình mới.

Bài tập 5: Nhấn giữ theo thời gian (Hold Time Detection)

- **Yêu cầu:** Viết thuật toán phân biệt giữa "nhấn nhả nhanh" và "nhấn giữ":
 - Nếu người dùng nhấn và nhả nút trong khoảng thời gian ngắn (dưới 2 giây), hệ thống ra lệnh tắt LED (hoặc giữ nguyên trạng thái tắt).
 - Nếu người dùng nhấn và giữ chặt nút liên tục quá 2 giây, hệ thống ra lệnh bật sáng LED.